Greedy Algorithms

Set Cover

$I = \{S_i, i = 1{:}m\}$& $\cup_{k=1:m} S_k = B, S_k \neq \{\phi\}$

SET COVER

Input: A set of elements B; sets $S_1, \ldots, S_m \subseteq B$

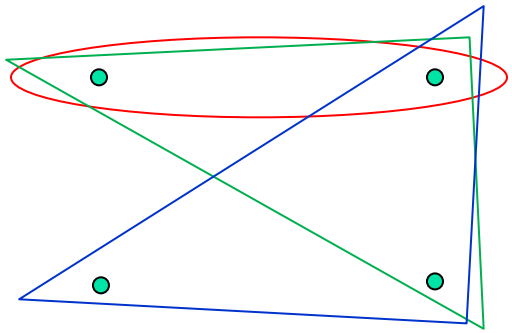
Output: A selection of the S_i whose union is B.

Cost: Number of sets picked.

Minimize $|I_s|$ *or* $|J| (\leq m), I_s = \{S_i, \cup_i S_i = B\}, J = \{i, \cup_i S_i = B\}$

Example: B is a universal set of topics covered by m available ALGO courses. S_j is the set of topics covered by the j^{th} ALGO course. Taking **minimum number of which ALGO courses** will allow us to learn all the topics being **covered by all the m of them**?

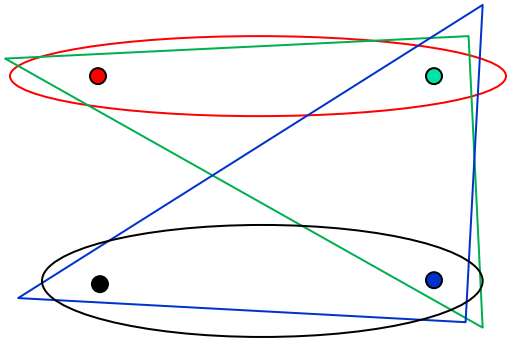
More than 1 optimal!



Courses

Topics

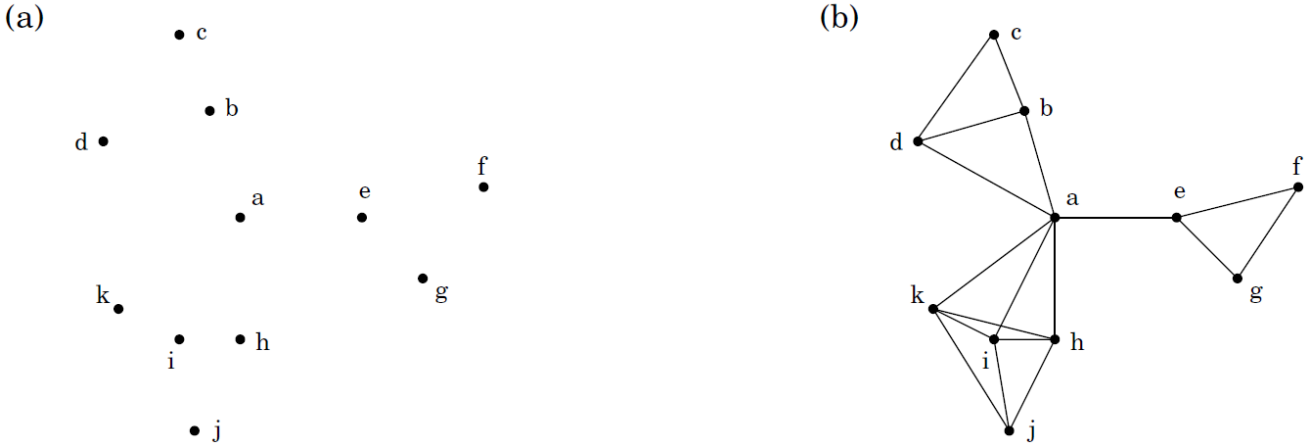
A graph? (number of points (n) = number of sets (m))



Example: B is a universal set of n towns. S_j is the set of towns (that will be) covered by the school of j^{th} town ($m = n$, symmetric, not transitive). Which **minimum number of schools** can cover all the n towns. [Vertex Cover]

A school in a town will cover another adjacent (edge) town

(a) Eleven towns. (b) Towns that are within 30 miles of each other.



Algo Skeleton:

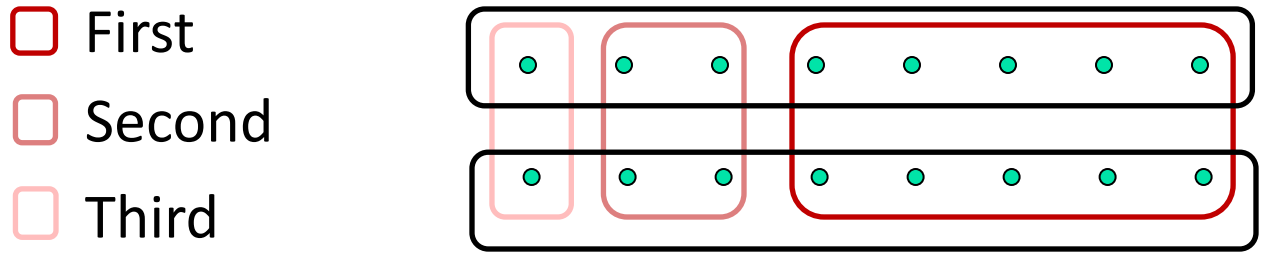
1. Set $J = \phi$
2. While $\cup_{i \in J} S_i \neq B$: Pick $i \notin J$ ($S_i \in I \backslash \text{Is}$) and add to J (I_s)
3. Return J (I_s)

Strategy?

Choose the largest uncovered set? $\max_{i \notin J} |S_i \backslash \cup_{k \in J} S_k|$

Valid as worst case will be all S_i chosen.

Optimal? NO.



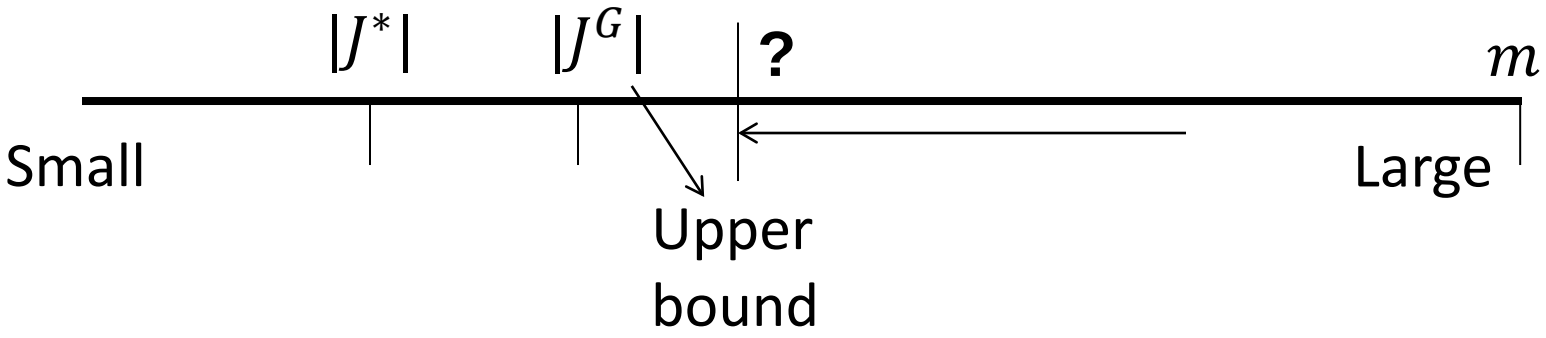
Time complexity!
$O(m^2 n)$

Greedy algos may not be optimal!

But usually Greedy algos are Polynomial!

Notion of approximation algos.

How close to optimal?



$n_t \rightarrow$ No. of uncovered points after t iterations

So $n_0 = |B|$ (n) $\qquad n_0 > n_1 > n_2 > \cdots > n_T$

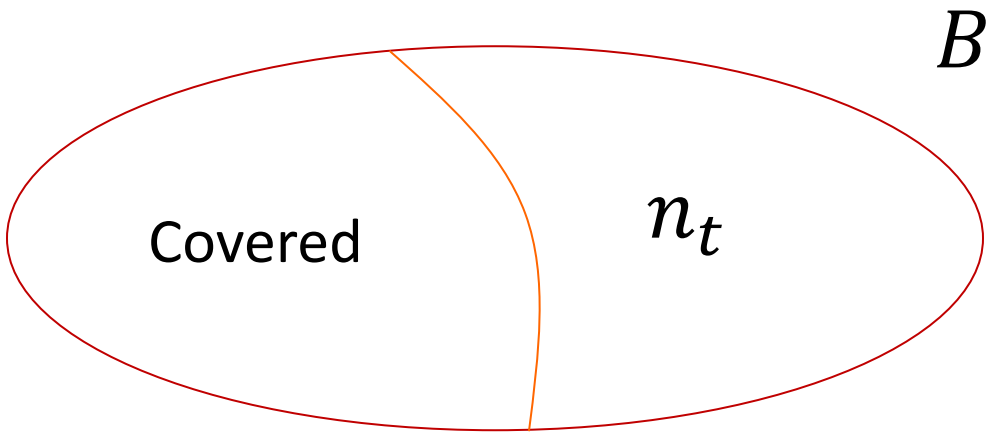
After some finite (T) iterations, we hope to get this small enough!

$$n_t \Big|_{t=T} < 1$$

Property: $n_{t+1} \leq n_t - \frac{n_t}{|J^*|}$

In the $(t+1)^{th}$ iteration, at least $\frac{n_t}{|J^*|}$ points are covered by the strategy of choosing the largest uncovered set

Proof:



$|J^*|$ many sets cover the entire B, that is, the n elements

Therefore, one set among the $|J^*|$ must have at least $n/|J^*|$ elements!

As at most all J^* sets cover the n_t elements, one set must have at least $n_t/|J^*|$ of the n_t elements!

The Strategy: Choose the largest uncovered set!

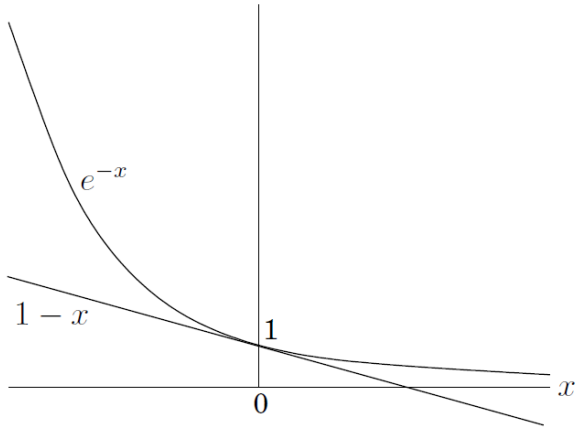
So at least $n_t/|J^*|$ of the n_t elements will be present in the chosen set!

Now:

$$n_t \leq n_{t-1} - \frac{n_{t-1}}{|J^*|} = n_{t-1}\left(1 - \frac{1}{|J^*|}\right)$$

$$\leq n_{t-2}\left(1 - \frac{1}{|J^*|}\right)^2 \leq n_0\left(1 - \frac{1}{|J^*|}\right)^t$$

Need to iterate till $n_t < 1$!



$$n_t \leq n_0\left(1 - \frac{1}{|J^*|}\right)^t$$

$$\leq n_0\left(\exp\left(-\frac{1}{|J^*|}\right)\right)^t$$

$n = n_0$. As for $|J^*| < \infty$, putting $t = |J^*|\log(n)$:

$$n_t < n\exp\left(-\frac{t}{|J^*|}\right) = n\exp(-\log n) = 1$$

At most number of sets to be chosen for complete coverage: The upper bound!

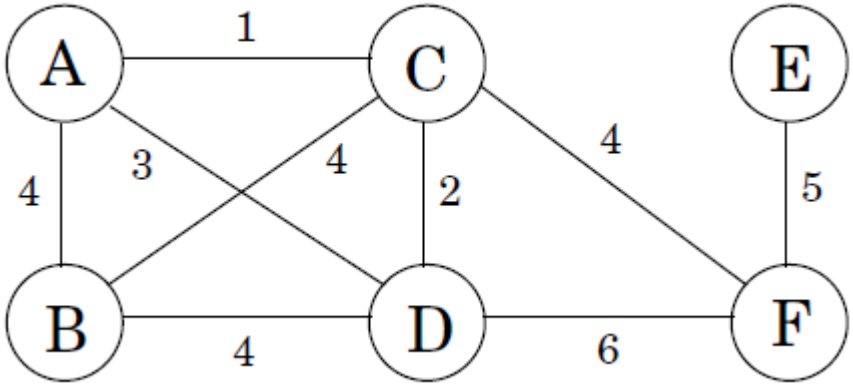
Approximation factor: Upper bound / Optimal $\leq \log n$

Minimum Spanning Trees (MST)

Input: Undirected graph $G = (V, E)$, weights $\omega: E \to \mathbb{Z}$

Output: an MST, that is, a "spanning" tree T in G that minimizes $\omega_T = \sum_{e \in E(T)} \omega(e) \,.\, \big(E(T) \subseteq E(G)\big).$

Spanning – touches all vertices in G



Weights (positive) are cost

cheapest possible network?

Without compromising connectivity!

Observations:

Removing a cycle edge cannot disconnect a graph

⟶ the optimal set of edges cannot contain a cycle

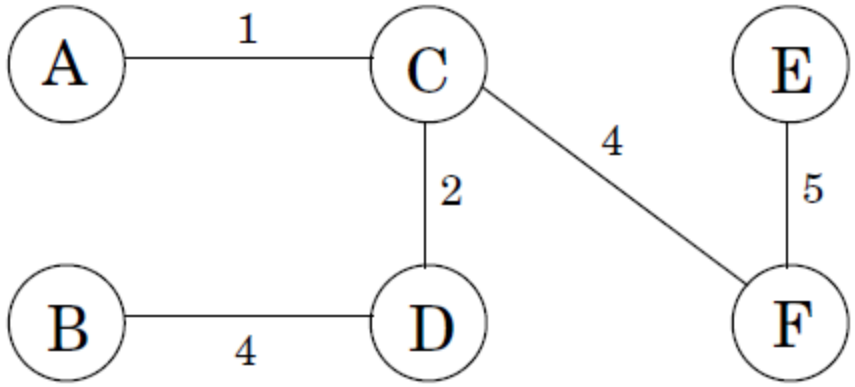
solution must be connected and acyclic → No 2 different paths between same nodes.

↓

undirected graphs ⟶ *trees*

Hence, Minimum Spanning Trees!

One optimal solution:
Cost (weight sum) = 16.



All possible MSTs will have the same number of edges, $|V| - 1$

Without loss of generality:

- All edge weights are considered non-negative (for ease of analysis)
- Negative edges can be made positive by adding a suitable large positive number to the all edges without effecting the MST outcome.

We are not searching for shortest path!

Trees: No roots needed!

Def: A tree is a connected acyclic graph.

Property: A tree (G) on n nodes has $n-1$ edges ($\mathrm{E}(G)$).

A connected graph with V vertices: if acyclic then has $|V|-1$ edges.

$|E(G)| < |V| - 1 \longrightarrow$ Not possible!

Lets start from an empty graph with V vertices and add edges one by one.

- Beginning: $|V|$ connected components & '0' edges
- At the end: 1 connected component
- At each iteration:

  Number of connected components $C \rightarrow C - 1$

  … as we should add the edge such that it does not create a cycle!

  So we will conclude exactly after $|V| - 1$ iterations adding $|V| - 1$ edges.

Property: Any connected undirected graph $G = (V, E)$ with $|\mathrm{E}| = |V| - 1$ is a tree.

A connected graph (G) with V vertices: if it has $|V| - 1$ edges, it is acyclic.

The converse to the previous property.

Lets start from the connected graph G.

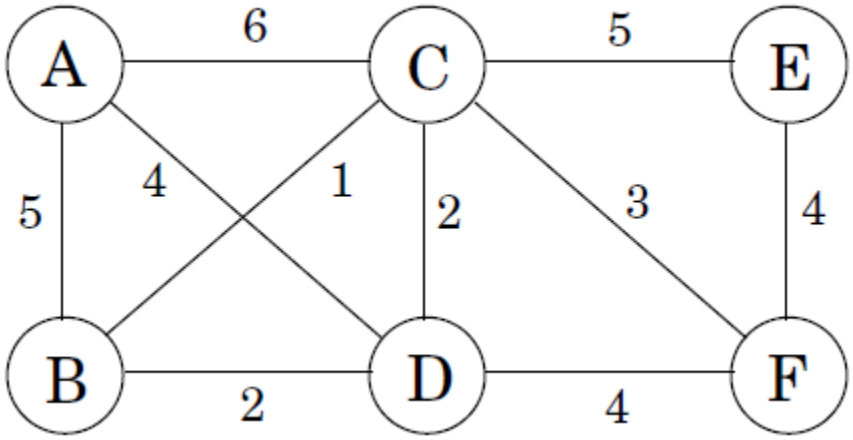
- While G has a cycle, remove an edge from that cycle. It will not affect connectivity.
- After previous step $G \rightarrow G'$. G' is obviously acyclic.
- G' has $|V| - 1$ edges [from previous property].
- So if G has $|V| - 1$ we cannot remove any edge, as essentially $G \equiv G'$. So such a G is acyclic.

Another…

Property: An undirected graph is a tree if and only if there is a unique path between any pair of nodes.

Tree ($G = (V, E)$) $\leftrightarrow$ A unique path exists between u & $v, \forall u, v \in V$

A Greedy Approach



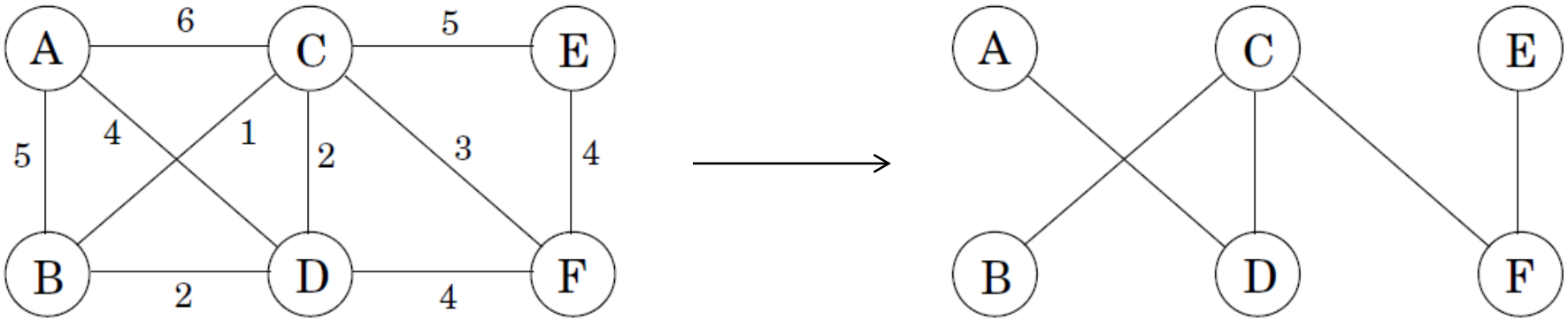
Lets consider the following greedy approach:

- Start with the empty graph.
- Repeatedly add the next lightest edge that doesn't produce a cycle.

↓

Ordering the edges!

$B-C,\ C-D,\ B \not\times D,\ C-F,\ D \not\times F,\ E-F,\ A-D,\ A \not\times B,\ C \not\times E,\ A \not\times C$

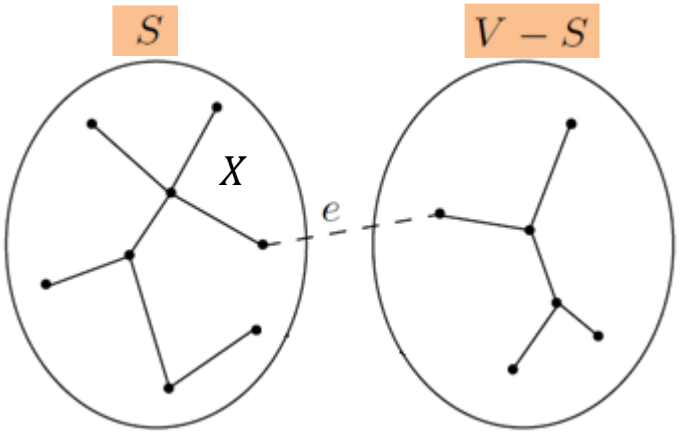


Correctness? Optimality?

The cut property

Cut def: A cut in G is any pair of the form $(S, V-S)$.

Property: The cheapest edge in any cut is in some MST.

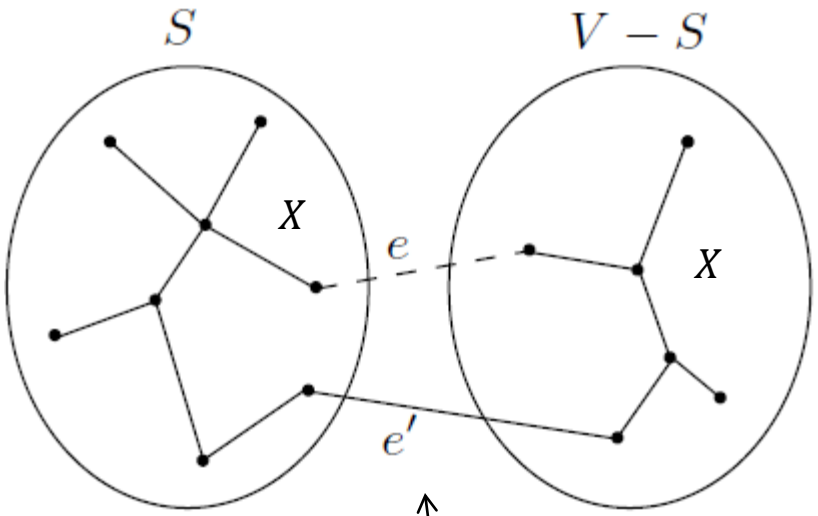


Cut property *Suppose edges X are part of a minimum spanning tree of $G = (V, E)$. Pick any subset of nodes S for which X does not cross between S and $V - S$, and let e be the lightest edge across this partition. Then $X \cup \{e\}$ is part of some MST.*

Proof:

Pick any MST: T

Assume that $e \notin T$



But the MST must connect the end points of *e*! So let e' be the edge in MST that does the crossing.

Now, get a T' by deleting e' from T and adding e.

T contains X but not e, and T' contains $X \cup \{e\}$.

- Is T' connected? Yes
  Broken parts of T (after e' deletion) are individually connected & e connects them!
  If e' was not deleted, it would have formed a cycle. Deleting an edge causing cycle does not affect connectivity!

- T' has $|V| - 1$ edges? Yes
  We removed 1 edge from an MST and added 1!

So T' is a tree!

Now, total weight of T' :

$$\omega_{T'} = \omega_T + \omega(e) - \omega(e')$$

But e is the cheapest / lightest (or one of the lightest) weight across the partition! So:

$$\omega_{T'} \leq \omega_T$$

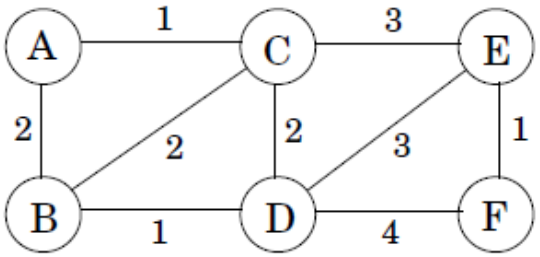
But as T is an MST we can only have: $\omega_{T'} = \omega_T$

And T' is an MST!

Hence, e and $X \cup \{e\}$, both are parts of the T' MST.

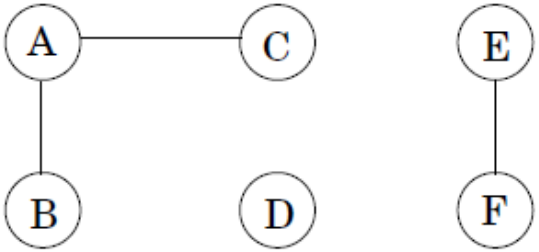
Figure The cut property at work. (a) An undirected graph. (b) Set X has three edges, and is part of the MST T on the right. (c) If $S = \{A, B, C, D\}$, then one of the minimum-weight edges across the cut $(S, V - S)$ is $e = \{D, E\}$. $X \cup \{e\}$ is part of MST T', shown on the right.

(a)

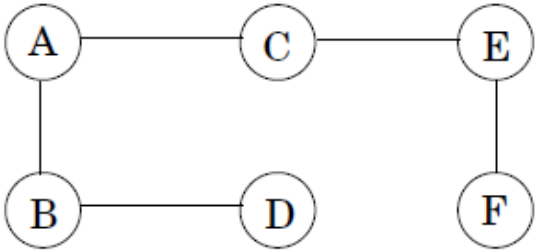


(b)

Edges X:

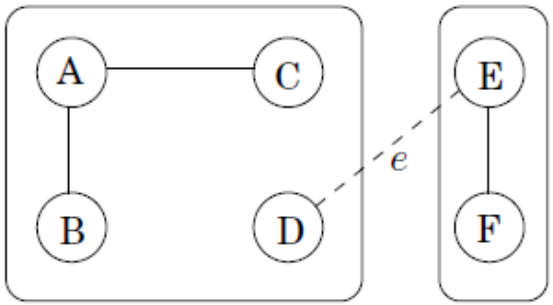


MST T:



(c)

The cut:



MST T':